

Known Issues

1. **RESOLVED: STM32 wireless/SD firmware updates now work reliably (Katapult bootloader)**

Summary

What works NOW	ESP32-P4 self-update from SD  · STM32 wireless update from ESP32  · STM32 SD update 
What USED to fail	STM32 update from SD/WiFi (pre-Katapult, ROM bootloader approach)
How it was fixed	Katapult bootloader — a persistent 8 KiB bootloader at <code>0x08000000</code> that handles UART firmware updates with CRC validation, no BOOT0/NRST pins required
Status	PRODUCTION-READY (commit <code>29be8a0</code> , 2026-06-21)
Details	See <code>docs/firmware/KATA-PULT_BOOTLOADER_SETUP.md</code>

What Katapult is and how it fixed the problem

Katapult (formerly CanBoot) is a **persistent bootloader** that sits in the first 8 KiB of flash (`0x08000000`) and runs on every boot. It checks for a software update request signature in RAM; if present, it stays in update mode and speaks a **simple, CRC-checked protocol over USART1 @ 250000 baud 8N1** — no parity, no BOOT0 pin required. If no request, it immediately jumps to the main app at `0x08002000` .

Why this solved everything:

1. **No BOOT0/NRST pins needed** — the app writes a magic signature to a known RAM address and warm-resets; Katapult sees the signature and stays in bootloader mode.
2. **No parity complexity** — Katapult uses 8N1 (same as the app), not the ROM's 8E1.
3. **CRC validation** — every block is CRC-checked; a corrupted flash is detected and rejected, not silently written.
4. **Un-brickable** — even a failed flash leaves Katapult intact; re-run the flash to recover. ST-LINK is now only needed for initial Katapult installation.

Flash methods now working:

- **Wireless via ESP32-P4:** upload `.bin` to `http://<esp-ip>/stm32` or run

```
python3 tools/stm32_flash_wifi.py
```

- **SD card:** copy firmware to SD, tap “Update STM32” on touchscreen
- **USB-to-serial from PC:** `python3 tools/katapult_flash_usb.py /dev/ttyUSB0 firmware.bin`

All three paths use the same Katapult protocol and work reliably in production.

What problems this document USED to describe (historical context)

Before Katapult (commits before `29be8a0` , June 2026), wireless STM32 flashing relied on the **STM32 factory ROM bootloader** (AN3155 protocol). That approach had insurmountable problems:

1. **BOOT0 pin not wired** — the ESP32-8048S043C (legacy board) had no free GPIO to drive STM32 BOOT0, required for ROM bootloader entry.
2. **Software-only jump failed** — the STM32G4 ROM bootloader would not respond after a warm jump from the app (silicon limitation).
3. **8E1 parity complexity** — ROM bootloader required even parity; switching mid-stream caused reliability issues.

Those problems led to the original workaround: **“use an ST-LINK to flash the STM32.”**

The ROM bootloader approach is documented in `docs/archive/WIRELESS_STM32_FLASH_EXPLAINED.md` (kept for historical reference). The Katapult approach replaced it wholesale.

Current recommendation

For STM32 firmware updates: use wireless/SD flashing via Katapult. See `docs/firmware/KATAPULT_BOOTLOADER_SETUP.md` for setup and usage.

ST-LINK is still useful for:

- Initial Katapult installation (one-time, flash `katapult-g474-usart1-pa9pa10.bin` to `0x08000000`)
- Deep debugging (breakpoints, memory inspection)
- Emergency recovery if Katapult itself somehow gets corrupted (rare)

But **ST-LINK is no longer required for routine firmware updates in the field.**

Other known issues

(No other major issues currently documented. This section will be populated as new issues are discovered.)